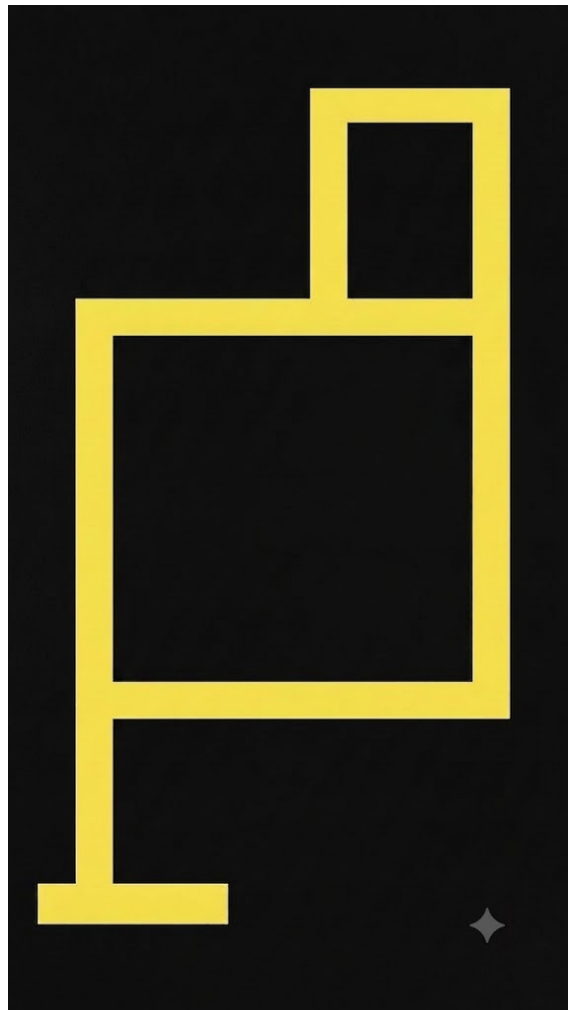


LEGO EV3

Line-Following Robot

A complete first-day setup, code review, simulator-backed Q-table, and hardware tuning guide

Prepared from the supplied map, Python files, Q tables, boot files, and colleague guides •
29 September 2026



What you will have by the end: the brick connection verified; sensors and motors checked; sensor readings calibrated; the starter Q table installed; a cautious first line-following run completed; and a saved copy of your tuned table.

1. Review findings and the correct starting point

I checked the supplied code, both Q tables, the map, and the environment notes. The original files are useful starting material, but the documentation mixes an older session's observations with your current robot. Use live checks tomorrow for network address, connected ports, lighting thresholds, and firmware version.

Item	Finding	Action for tomorrow
Map	Yellow path on a dark background; a large loop and a smaller upper loop/branch are visible. No start arrow or required direction is marked.	Confirm the start point and direction with your course task. A one-sensor robot cannot choose a branch by intention without extra junction logic.
main.py	Syntax compiles under CPython, but imports EV3-only Pybricks modules; no device execution was possible here. Bottom-level TRAINING=True starts training before run().	Use TRAINING=False for the first table test. Check code paths, sensor values, and stop behavior before any exploration run.
q_table.pkl	Despite its extension, this is a plain Python dictionary literal with 18 entries (2 modes × 3 sensor classes × 3 actions).	Its greedy actions match the simulated policy, but values and sensor thresholds are not verified against this exact mat.
q_table_v3.pkl	A CPython protocol-4 pickle. EV3 MicroPython compatibility is not established.	Do not use as the starting file. Convert the provided text seed on the brick with the included helper.
make_qtable.py	Reads q_table.pkl as text and overwrites that same path with binary pickle. The second run can fail because the input is now binary.	Use make_qtable_ev3.py and q_table_simulated.txt; the source stays separate.
Boot files	u-boot(1).bin and boot(1).scr are boot artifacts. uEnv is mostly commented settings.	Do not flash or replace boot files for this project. Application setup does not require it.

Important sensor assumption

The current program gives +10 only when the reading is MIDDLE (reflection 9–24) and calls that “on the line.” The map shows a bright yellow strip against a dark surface; yellow may read above 25 and therefore be classified WHITE. Sensor naming is only a label. Measure the actual readings first and decide which class means “on the yellow tape.” Until then, the simulator table is a starting policy for the code's existing rules, not proof of correct hardware tracking.

2. Understand the controller before powering the motors

The program has one reflected-light sensor, two drive motors, and an infrared sensor. It quantizes one reflection value into WHITE, MIDDLE, or BLACK, and adds a boolean mode to remember which side/direction it is correcting. The Q table stores one value per (mode, light class, action).

Code setting	Current value	Meaning / question to verify
Motors	A and D	Left motor on A and right motor on D. Check actual cable ports and orientation.
Sensors	Color S1; IR S4	Check the live port scan; do not rely on a colleague's earlier port report.
DriveBase	wheel diameter 40 mm; axle track 50 mm	These affect distance/turning; verify against wheel measurements and the actual build.
Reflection thresholds	WHITE ≥ 25 ; BLACK ≤ 8 ; otherwise MIDDLE	Uncalibrated. Yellow-on-black may not produce a useful MIDDLE band.
Actions	forward for 250 ms; left/right turning until class changes	Turning loops have no timeout. If sensor readings do not change, the motor can keep turning.
Reward	+10 for new MIDDLE; -10 for new WHITE/BLACK	Appropriate only if MIDDLE really represents the desired line state.
Exploration	$\epsilon = 2.7321^{-(\text{iteration}/1000)}$, stop near $\epsilon < 0.01$	Exploration is hazardous on a physical course. Do short supervised sessions only after baseline operation.

Files included in the companion bundle

- main.py — your supplied EV3 application, preserved for review.
- q_table_simulated.txt — readable 18-entry starter table produced by the simulator.
- simulate_qtable.py — reproducibly runs the abstract model and writes the seed table.
- make_qtable_ev3.py — converts the trusted text table to the pickle structure that the supplied main.py expects.
- Map.jpeg — your route image.
- REVIEW_NOTES.md — short handoff notes.

3. What the simulator did—and did not do

The supplied code has two mode-specific transition patterns in `m_x` and `m_y`. I used those rules to define a small deterministic model over the same 18 (mode, sensor class, action) combinations. The model rewards MIDDLE, penalizes WHITE/BLACK, keeps the mode fixed for the listed corrections, and leaves unlisted transitions in the same light class. I ran 300,000 seeded Q-learning transitions (seed 292026; $\alpha=0.10$; $\gamma=0.90$) and confirmed all 18 numeric values and the greedy policy.

Mode	Reading	Best action in seed	Q-value
True	WHITE	turn_left	100
True	MIDDLE	forward	100
True	BLACK	turn_right	100
False	WHITE	turn_right	100
False	MIDDLE	forward	100
False	BLACK	turn_left	100

This result agrees with the original table's greedy actions for every one of the six states. The old table therefore looks internally consistent with the same assumed transition pattern. The simulated numerical values are higher because the model makes deterministic, optimistic transitions. That does not mean the simulated values are better on the real robot.

Simulator boundary

This is not a camera/physics simulation of `Map.jpeg`. The map has no stated dimensions, start orientation, route direction, wheel slip, sensor offset, tape reflectance, or junction rule. The current state representation does not encode position or route choice. Consequently, no honest simulator can establish that a single-sensor controller will navigate the full map correctly from the picture alone. The table is a warm start so tomorrow's work can focus on calibration and short fine-tuning, rather than a cold zero table.

What would make a map-level simulator possible?

- A marked start point and travel direction; clarify which loop/branch to take at each junction.
- Measured path dimensions, tape width, sensor location relative to wheel axle, and wheel circumference/track.
- Reflection samples for yellow tape, dark mat, and tape edges in the room lighting.
- An explicit junction/route policy, or additional sensors/heading history.

The map image shows one yellow connected path with loop/branch geometry. A line-following Q table can learn local steering states; it does not automatically encode which branch to select at a junction.

4. Tomorrow: ordered setup and first run

Work in this order. Do not skip ahead to exploration. Keep the robot on a clear, flat surface; lift the driven wheels for the first motor-direction test. Have the map, robot, USB/network connection, and laptop ready.

Step 1 — identify the live brick and connection

- Power on the EV3 and wait for its boot to finish. Confirm the brick display is responsive and the battery is charged.
- Connect the PC and brick to the same network, or use the direct link-local connection you used previously. The supplied address 169.254.166.35 can change; read the current address from the brick/network settings.
- On Windows, open PowerShell and run `ipconfig`. Identify the adapter that connects to the brick; a 169.254.x.x address is a link-local address. Use the brick's current address, not a stale one from a note.
- Check reachability, then authenticate interactively:

```
ping 169.254.166.35
ssh robot@169.254.166.35
# enter the robot password only when prompted
```

If mDNS works on your network, `ssh robot@ev3dev.local` may work. The colleague guide reports robot/maker and an EV3Dev 2.3.5 kernel, but those are historic observations, not guaranteed current settings.

Step 2 — verify firmware and hardware ports

Once connected over SSH, inspect the OS and devices. In SSH run:

```
uname -a
cat /etc/os-release
for d in /sys/class/lego-sensor/*; do
    echo -n "$d "; cat "$d/address"; cat "$d/driver_name"
done
for d in /sys/class/tacho-motor/*; do
    echo -n "$d "; cat "$d/address"; cat "$d/driver_name"
done
```

Compare the results to the source: color sensor in1/S1, IR sensor in4/S4, large motors outA/outD. If anything differs, correct the Port constants only after writing down the actual layout. The source expects the Pybricks MicroPython APIs, so check that ``pybricks-micropython`` and ``brickrun`` exist before transferring.

Step 3 — inspect the code and table on the laptop

In PowerShell, go to the folder containing the downloaded companion files. Use a local Python 3 installation to check syntax and the simulator output:

```
py -3 -m py_compile main.py simulate_qtable.py make_qtable_ev3.py
py -3 simulate_qtable.py
```

Expected simulator summary: 300,000 transitions, 18 entries, and these best actions: mode True: WHITE→turn_left, MIDDLE→forward, BLACK→turn_right; mode False reverses left/right. The generated values are recorded in q_table_simulated.txt. The main.py check is only a syntax check; EV3 modules are absent from normal Windows Python, so this does not prove device execution.

Step 4 — calibrate the actual surface

Make a small color-sensor-only calibration program or temporarily add a diagnostic loop. Put the robot in a safe position with the sensor at its normal height. Record 10–20 reflection values over each surface: yellow tape, dark background, and the tape edge. Use the median or typical range, not one reading. Repeat after the room lights are on.

```
from pybricks.ev3devices import ColorSensor
from pybricks.parameters import Port
from pybricks.tools import wait
sensor = ColorSensor(Port.S1)
for i in range(50):
    print(sensor.reflection())
    wait(200)
```

Run it from the brick project folder with `brickrun -r -- pybricks-micropython calibrate_sensor.py` if the brick requires the graphics runtime. Keep the reading record. Decide whether bright yellow maps to WHITE or MIDDLE under the thresholds. If yellow is consistently ≥ 25 , the existing reward is probably rewarding the dark/edge class, not the tape; change the thresholds/reward/state definitions before trusting any Q values.

Step 5 — upload only application files

Make the project directory first if needed; use SFTP/SCP to transfer main.py, q_table_simulated.txt, and make_qtable_ev3.py. The Q table source should remain a separate text file. Example with Windows OpenSSH from the local project directory:

```
scp main.py q_table_simulated.txt make_qtable_ev3.py robot@169.254.166.35:/home/robot/
```

The provided Paramiko line uploads `C:\...\run.py` to `/home/robot/seniru/run.py`. It does not match the attached main.py name and assumes the remote folder already exists. Either create that directory and intentionally rename the file, or use the simpler `/home/robot/` path above. Avoid putting passwords in scripts or command history; prompt for them or use an SSH key.

Step 6 — build the EV3-compatible table

SSH into the brick, change to the upload folder, and execute the helper with the same launcher used by the project:

```
ssh robot@169.254.166.35
cd /home/robot
brickrun -r -- pybricks-micropython make_qtable_ev3.py
ls -l q_table_simulated.txt q_table.pkl
```

Expected message: “Saved 18 entries to q_table.pkl”. This creates the binary table from the separate, trusted text source; the source is not overwritten. The helper uses `exec` to parse the generated literal, so only use the generated file from this bundle, never an unknown table source.

Step 7 — set safe runtime mode and check startup

In `main.py`, set the final runtime flag to `TRAINING = False` for the first test. Do not change the `TRAINING = False` local assignment inside `learn()`; it is a separate stop-condition variable. Upload the edited file. Confirm the intended working directory contains `main.py` and `q_table.pkl`. The current `main.py` calls `run()` after `learn()` returns, so `TRAINING=True` means “train, then immediately enter line-following,” not “train only.”

```
scp main.py robot@169.254.166.35:/home/robot/
ssh robot@169.254.166.35
cd /home/robot
brickrun -r -- pybricks-micropython main.py
```

For a remote run, start foreground first. Watch the brick and keep access to Ctrl+C. Avoid `nohup` until the foreground run is reliable and you know how to stop it. If a turn loop gets stuck, stop the process immediately.

Step 8 — test motion in a controlled sequence

- With wheels lifted, verify that forward moves the robot forward and that left/right pivots match the names. If not, correct motor port order or signs before putting it on the mat.
- Place it at a straight section, sensor centered over the tape. Confirm the sensor class on yellow and dark surface from your recorded readings.
- Run only the preloaded policy; observe 10–20 seconds, then stop and inspect. Check whether it stays on the yellow tape, especially through bends.
- Do not test obstacle avoidance until line tracking is stable. The IR sensor and beacon configuration must be separately validated; the current distance threshold is 15 cm.

5. Fine-tune without starting from zero

Keep a copy of the seed table before each test. The provided simulator seed already has a deterministic policy in all six mode/sensor states. Fine-tuning means checking sensor classes and transition behavior, then adjusting a small number of values or running a short supervised learning session from this table. Do not expect the table to compensate for incorrect calibration or an unresolved junction choice.

Recommended tuning loop

- 1. Copy `q_table_simulated.txt` to a dated backup on the PC. Record the exact thresholds and measured readings.
- 2. Test the greedy policy with `TRAINING=False` on one straight and one curve. Record each encountered mode, sensor class, chosen action, and whether the robot moved toward the tape.
- 3. If turning direction is wrong, check left/right motor wiring and the mode assignment first. Do not “fix” a wiring problem by changing Q values.
- 4. If the sensor class is wrong, revise thresholds and reward semantics. Re-run the simulator with the revised assumptions; do not train against a mislabeled state.
- 5. Only after baseline behavior is stable, set `TRAINING=True` for a short supervised session on a wide, low-risk track section. Keep a hand ready to stop.
- 6. Save and retrieve the resulting `q_table.pkl` under a new filename after each run. Do not overwrite your seed or only copy.
- 7. Compare the new greedy actions to the last known-good table. Keep the candidate only if the robot improves in repeated trials.

Test record field	Example
Date / map version	29 Sep 2026 / Map.jpeg
Thresholds	BLACK $\leq$ __ ; MIDDLE __-__ ; WHITE $\geq$ __
Measured medians	Yellow __ ; dark mat __ ; tape edge __
Start and direction	Mark on map / clockwise or counterclockwise
Observed failure	Exact location, sensor class, mode, action
Change and result	One change at a time; repeat at least 3 runs

Code risks to address before longer training

Risk in supplied main.py	Why it matters / mitigation
Unbounded turn_left/turn_right loops	If the reading never changes, motor keeps turning. Add a maximum time/iteration guard and a stop/recovery behavior before long sessions.
TRAINING=True by default	A fresh run explores randomly. Set False for first deployment, and add a physical emergency stop procedure.
Save pickle every iteration	Repeated flash writes can slow training and wear storage. Consider saving every N updates plus on clean stop after initial tests.
No junction handling	At a branch/crossing a single reflection state does not encode desired route. Add a defined junction state/policy or change sensing hardware.
IR fallback flips mode unconditionally	backward() turns to find MIDDLE, but then run() does mode = not mode without verifying orientation. Check this behavior with obstacles disabled first.
Calibration constants assume another surface	Tune from measured yellow/dark samples and validate state interpretation before using table rewards.

6. Troubleshooting and finish checklist

Use the symptom to choose the next check. Change one variable at a time.

Symptom	likely cause	check
SSH times out	Wrong/stale IP, adapter, or network	Read current brick address; confirm PC interface and ping.
ENODEV at startup	Port mismatch or loose cable	Run sysfs port scan; reconnect sensor/motor; update Port constants if needed.
Could not initialize graphics	Wrong launcher	Run with brickrun -r -- pybricks-micropython.
Unicode/decode error loading Q table	PC protocol-4 pickle or text/binary mix-up	Keep source text separate; run make_qtable_ev3.py on the brick.
Immediate turn loop	Sensor never crosses threshold or wrong turn action	Stop; read values; inspect thresholds, sensor height, tape, motor wiring. Add timeouts before repeat.
Robot turns away from yellow	State labels/reward or motor orientation reversed	Compare actual readings and action direction; fix calibration or motor mapping first.
Robot chooses wrong map branch	No route memory/junction state	Current policy is only line steering. Specify route and implement junction behavior.

Before leaving the lab

- The brick address, OS version, verified ports, and sensor readings are written down.
- The wheels-forward and turn direction checks pass.
- The Q-table has exactly 18 entries and the on-brick helper reports success.
- The first run used TRAINING=False and was observed closely.
- At least one known-good table is copied back to the PC with a dated name.
- Any run process is stopped cleanly; robot, cables, and boot files are left unchanged.

Files and commands summary

```
Local folder: main.py, q_table_simulated.txt, make_qtable_ev3.py
On-brick folder: /home/robot/
Build Q table: brickrun -r -- pybricks-micropython make_qtable_ev3.py
Run baseline: brickrun -r -- pybricks-micropython main.py
Copy trained table back (PowerShell):
scp robot@<CURRENT_IP>:/home/robot/q_table.pkl ./q_table_after_test_01.pkl
```

Scope note: This guide checks code structure and simulates the controller's abstract state/action rules. It does not claim a physical EV3 run or a full geometry/physics simulation; those require the live brick and measured track.